

Class 8 — Intrusion Detection and Prevention Systems

Writing rules · catching injection attempts · proving the sensor detects

Wednesday, September 16, 2026 — 6:00 pm EDT (Eastern Daylight Time)

LAB_04 — IDS/IPS Configuration and Validation (part 2 of 2)

Cyber.SoHo Educational Hub — From the Industry to the Classroom



Before we start — legal notice and acceptable use (1/3)

Educational purpose only. The Material shows how to protect systems, not how to attack the systems of others. Nothing in it is an invitation, an instruction, or an authorization to access, test, or damage any real system, network, account, or data.

YOU MAY

- Practice **only inside your personal, isolated laboratory** — VirtualBox, or your own free-tier AWS or Microsoft Azure account (LAB_01).
- Practice on any other system **only with explicit written permission** from its owner, obtained before you start.
- Use **snapshots and isolated virtual networks** so that a mistake stays inside your laboratory.

YOU MUST NEVER

- **Never** scan, probe, attack, intercept, monitor or access any system, network, wireless network or account that is not yours — college, employer, business, public website, neighbour.
- **Never** create, install or release malicious software outside your isolated laboratory.
- **Never** attempt password, credential or interception attacks against any real account or communication.
- **Never** use the Material or any tool in it to break a law or a policy that applies to you.

By reading, watching, downloading or using any part of the Material you accept the entire Legal Notice. Enrolled students also sign the separate Acknowledgment and Agreement page at the beginning of each session.



Legal warning, cloud responsibilities, ethics and AI (2/3)

Legal warning

Criminal Code of Canada: s. 342.1 unauthorized use of a computer · s. 342.2 possession of a device to obtain unauthorized use · s. 430(1.1) mischief in relation to computer data. **Penalties can include imprisonment.**

Outside Canada, the laws of your own country, province or state apply. College policies and codes of conduct apply in addition.

Also cited in this course: Québec Law 25 · PIPEDA · GDPR · US CFAA 18 U.S.C. § 1030 · EU Directive 2013/40/EU · Spain Ley Orgánica 10/1995.

The instructor never asks for, and never authorizes, any illegal act

If an activity seems to touch a system you do not own, stop and ask the instructor before doing anything.

The injection strings of Lesson 2 are typed into **your own lab web server only**, to prove that **your own sensor** sees them.

Cloud responsibilities

Respect the acceptable-use policies of AWS or Microsoft Azure.

Free tiers have limits — monitor them.

All cloud costs, overage charges and account consequences are your responsibility.

Ethics and responsible conduct

Defensive ("white-hat") skills: protect, never harm; respect the privacy of others.

A real vulnerability found by accident is **reported responsibly** to the system owner and to the instructor — never exploited or published.

Academic integrity — the AI rule

Any AI tool used in a submission: state the **LLM** and the **exact prompt(s)**, in every chapter concerned.

"The model drafts. The analyst decides. The analyst's name goes on the record."



Warranty, liability, property, privacy and acceptance (3/3)

- **Assumption of risk, no warranty, no professional advice** — you use the Material at your own risk, "as is" and "as available"; it is not legal advice and not professional security consulting.
- **Limitation of liability and indemnification** — Cyber.SoHo and the instructor are not responsible for damage, data loss, failures, suspensions, costs or legal consequences of use or misuse; you hold them harmless for your own actions.
- **Intellectual property** — © 2026 Cyber.SoHo, all rights reserved. Students hold a limited, personal, non-transferable licence: no redistribution, re-publication, resale, public posting or commercial use. **Your labs, project and manual belong to you.**
- **No affiliation** — product names (VirtualBox, AWS, Azure, Windows, Linux, Snort, Suricata, pfSense, OPNsense, WireGuard, OpenVPN, OpenSSL, CIS, YouTube, TikTok) are trademarks of their owners, used for identification only.
- **External links** — convenience only, not endorsements; verify a link before opening it (VirusTotal URL scanner).
- **Social media** — course clips on YouTube or TikTok are demonstrations inside an isolated laboratory; do not reproduce them outside such a laboratory. Third-party re-uploads and comments are not endorsed.
- **Privacy** — the Material collects no personal data. The session signature log records your name, session number, date and a verification code; keep real personal data out of your own deliverables.
- **Governing law** — Province of Quebec, Canada; severability; Cyber.SoHo may update the Notice, the current version replacing all previous ones.
- **Acceptance** — using any part of the Material means you have read, understood and accepted the Notice; enrolled students also sign the **Acknowledgment and Agreement** e-signature page at the beginning of each session (provided separately).



Tonight in one hour — three lessons, one lab

L1

Writing and reading simple IDS rules and alerts

The header, the options, the alert line — and the four mistakes that make a rule silent.

L2

Web application security basics

What an injection attack is, why it works, the developer's fix, and why a network sensor still helps.

L3

Validating that a detection system actually detects

Safe, contained test traffic; expected versus observed; negative tests; evidence for Chapter 4.

6:00

Recall

6:05

Lesson 1

6:25

Lesson 2

6:42

Lesson 3

7:00

LAB_04 briefing

7:30

Break

7:45

Lab work

Each lesson in the module: video · real-life example with runnable code · schematic · reference table · flash cards · quiz



Recall of Class 7 — the sensor is running; three ideas carry over

IDS versus IPS

An **IDS** (Intrusion Detection System) **detects and reports**.

An **IPS** (Intrusion Prevention System) sits **inline** and can **block**.

Tonight: action **alert** everywhere — evidence, not interruptions.

NIDS versus HIDS

A **NIDS** (network-based) reads copies of packets on a link.

A **HIDS** (host-based) reads what happens on one computer.

LAB_04 uses a NIDS placed where it sees the traffic **between your zones**.

HOME_NET

The variable that tells the engine **which addresses are ours**.

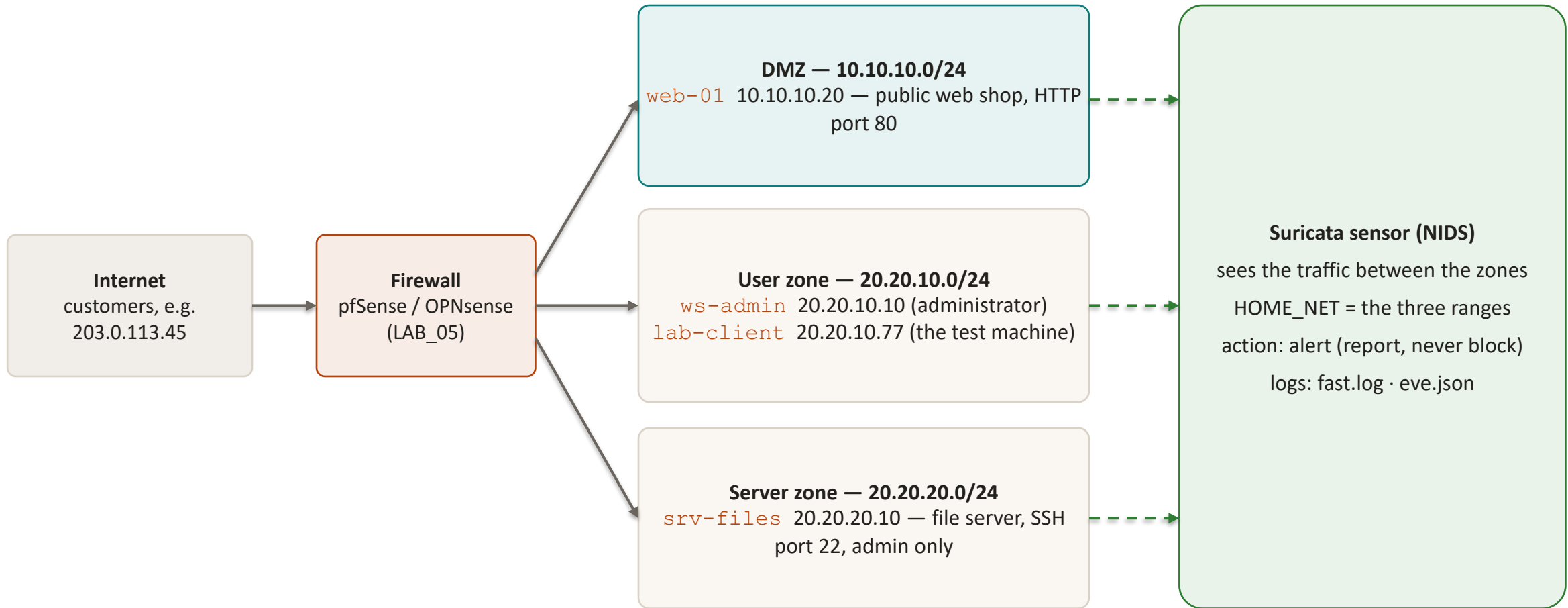
-> **\$HOME_NET** means "traffic going into our company".

Wrong HOME_NET = half the rules go silent — **the first thing to check** when an alert is missing.

Yesterday: one free, open-source sensor — Suricata 8.x or Snort 3 — installed on a lab machine, running and seeing traffic. Tonight: rules, alerts, and proof.



The company we defend — Laurentide Outfitters inc.



Sensor: Suricata 8.x (Snort 3 accepted — syntax differences shown where they matter). Addresses are illustration values: use your own LAB_01 zoning table for HOME_NET.



What a rule is — one line, two instructions

- A **rule** — also called a **signature** — is **one line of text**.
- Instruction 1, the **header: which packets to examine** (who, where, which protocol).
- Instruction 2, the **options: what to look for** inside them, and how the alert is named.
- When both are true for the same packet, the engine performs the **action** written first.
- Suricata and Snort share the same rule language; the small differences are pointed out where they matter.

THE HEADER — who and where

```
alert http any any -> 10.10.10.20 80
```

DETECTION OPTIONS — what to look for

```
( flow:to_server,established;  
http.uri;  content:"or 1=1";  nocase;
```

BOOKKEEPING — how the alert is named

```
msg:"LAB rule 4 - SQL tautology OR  
1=1 in a request to the web shop";  
classtype:web-application-attack;  
sid:1000004; rev:2; )
```

Tonight's specimen is rule 4 of the LAB_04 set. Header first, options second, alert only if everything is true.



The rule header (1/2) — seven fields, always in the same order

#	Field	Meaning	Values you will use
1	action	What the engine does when the whole rule is true	alert · drop (inline IPS only) · pass · reject
2	protocol	The protocol the packet must use	tcp · udp · icmp · ip · Suricata app-layer: http, dns, tls, ssh
3	source address	Where the packet comes from	IP, CIDR network, \$HOME_NET / \$EXTERNAL_NET, list, !negation, any
4	source port	Port on the source side	number · any · range 1024:65535 · !negation · \$HTTP_PORTS
5	direction	The way the packet travels	-> one direction only · <> both directions · there is no <-
6	destination address	Where the packet goes	same forms as the source address
7	destination port	Port on the destination side	same forms as the source port · 80 = HTTP · 22 = SSH

The header does the **cheap work** — addresses, ports and protocol are compared first; only packets that survive the header are examined for content.



The rule header (2/2) — the forms an address, a port and a direction can take

Addresses

a single address `10.10.10.20`
a network in CIDR notation
`20.20.20.0/24`
a variable `$HOME_NET $EXTERNAL_NET`
a list `[10.10.10.20,10.10.10.21]`
a negation `!20.20.10.10` (every address but this one)
anything `any`

Ports

a number `22 80`
anything `any`
a range `1024:65535`
a negation `!22`
a variable `$HTTP_PORTS`
application ports: 80 = HTTP, 22 = SSH, 443 = HTTPS (encrypted — content cannot be read)

Direction operator

`->` from source to destination only
`<>` both directions
there is **no** `<-` — swap the two sides instead
A test that goes the other way never matches a one-way rule (mistake 2 of § 1.4).

Rule 2 of tonight uses three of these forms at once: `!20.20.10.10 any -> 20.20.20.0/24 22`



The rule options (1/2) — bookkeeping: naming and numbering the alert

- `msg:"..."`; — the **message**: the human-readable text printed in the alert line. Write it so a colleague understands the event.
- `sid:1000004`; — the **SID** (Signature Identifier), **unique in the whole rule set**. Below 1,000,000 is reserved for distributed rule sets (ET Open); **your own rules start at 1,000,000**.
- `rev:2`; — the **revision**: a version counter you increase at every change of the rule text. Printed in the alert, so evidence names the exact version.
- `classtype:web-application-attack`; — the **classification**: a category from `classification.config`, which also assigns the **priority** (1 = highest).
- `reference:cwe,89`; — optional pointer to public documentation of the weakness.

RULE 4 — BOOKKEEPING PART

```
msg:"LAB rule 4 - SQL tautology OR 1=1  
in a request to the web shop";  
classtype:web-application-attack;  
sid:1000004;  
rev:2;
```

Every option ends with a **semicolon**.

A missing `;` is the most frequent syntax error — the engine refuses the rule and prints the line number.



The rule options (2/2) — detection: what must be true inside the packet

Option	What it requires	Used tonight in
<code>content:"or 1=1";</code>	The byte string that must appear. Case-sensitive by default.	rules 3 and 4
<code>nocase;</code>	Written right after a content: that content ignores letter case (OR 1=1, Or 1=1, or 1=1 all match).	rules 3 and 4
<code>http.uri;</code>	Sticky buffer: the following content options search only the request URI — normalized (%20 decoded to a space).	rules 3 and 4
<code>flow:to_server,established;</code>	Packet from client to server, inside a connection whose TCP handshake is complete. Removes server replies and half-open attempts.	rules 3 and 4
<code>itype:8;</code>	ICMP message type 8 = echo request (the outgoing half of ping). Type 0 = echo reply.	rule 1
<code>flags:S;</code>	TCP SYN flag set and no other: the first packet of a new connection — one alert per attempt.	rule 2

Options are checked **in the order written** — `http.uri` must come before the `content` it applies to.



Suricata versus Snort 3 — the one syntax difference you will meet tonight

Suricata

```
http.uri; content:"UNION SELECT"; nocase;  
buffer and modifier are separate keywords  
test the rules: suricata -T -c  
/etc/suricata/suricata.yaml
```

Snort 3

```
http_uri; content:"UNION SELECT",nocase;  
buffer with an underscore; modifier inside the content option, after a  
comma  
test the rules: snort -T -c snort.lua -R local.rules
```

- Everything else in the four LAB_04 rules is **identical** in both engines.
- `-T` means "test the configuration and stop" — run it after every edit, before restarting the engine.
- Local rules live in `local.rules`: listed under `rule-files` in `suricata.yaml`, or passed with `-R` to Snort 3.



How the engine reads a rule — header first, options second, all or nothing

Anatomy of one IDS rule

```
alert http any any -> 10.10.10.20 80 ( msg:"..."; flow:..., http.uri; content:"or 1=1"; nocase; sid:1000004; rev:2; )
```

THE HEADER — who and where

action

alert
log & keep

protocol

http
on TCP 80

source → dest

any → 10.10.10.20
the web server

ports

any → 80
HTTP

THE OPTIONS — what to look for

flow / http.uri

to_server
look in the URL

content + nocase

"or 1=1"
any letter case

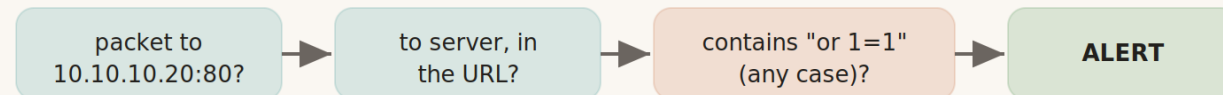
sid:1000004

unique id — 1,000,000+ for your own rules

rev:2

version of the rule

How the engine reads it: header first, then options — alert only if ALL are true



If any test is false the packet passes silently. That is why one wrong field (a wrong port, HOME_NET, or direction) makes a rule "never". Read the alert line back the same way: [1:1000004:2] is gid:sid:rev — it names exactly which rule fired.

420-D03-AB Cyber Defense · Class 8 · Lesson 1 · Cyber.SoHo Educational Hub

- **Header first** — cheap numeric comparisons.
- **Options second** — in the order written.
- **Alert only if ALL conditions are true.**
- Any false condition → the packet passes **silently**, nothing is logged, next rule.
- The alert line is read back the same way: [1:1000004:2] = gid : sid : rev.



The four mistakes that make a rule "never fire"

1 · Wrong HOME_NET

A rule written with `-> $HOME_NET` cannot match if the variable in `suricata.yaml` (or `snort.lua`) does not contain the destination of your test.

Compare the variable with your **LAB_01 zoning table** before anything else.

2 · Wrong direction or wrong port

A test going from the server back to the client does not match `->` written the other way.

An **HTTPS** request on port 443 can never match a `content` rule — the engine sees only encrypted bytes.

3 · Rule not loaded

`local.rules` must be listed under `rule-files:` (or passed with `-R`), the engine **restarted or reloaded** after every edit, and the identifier must not collide.

Read the **"rules loaded"** line at start-up.

4 · Case and encoding

Without `nocase`, `union select` does not match `content:"UNION SELECT"`.

Without `http.uri`, the engine may search raw bytes where the attacker wrote `UNION%20SELECT`.



Reading an alert line — fast.log, field by field

<code>09/16/2026-19:20:42.990001</code>	<code>[**] [1:1000004:2]</code>	<code>LAB rule 4 - SQL tautology OR 1=1 in a request to the web shop</code>	<code>[Classification: Web Application Attack] [Priority: 1]</code>	<code>{TCP} 20.20.10.77:441 21 -> 10.10.10.20:80</code>
timestamp — local time of the sensor, with microseconds	gid : sid : rev — generator 1 is the rule engine; rule 1000004, revision 2	the msg text of the rule, exactly	from classtype, through classification.config	protocol, source, destination of the packet that matched

- `/var/log/suricata/fast.log` — one plain-text line per alert (shown above). `eve.json` — one JSON object per event, with many more fields (the EVE log). Snort 3: `-A alert_fast` prints the same line.
- Watch alerts arrive while you test: `sudo tail -f /var/log/suricata/fast.log`
- Count alerts by rule for the manual: `grep -c 1000004 /var/log/suricata/fast.log`
- Read the addresses against your diagram: **the lab client sent a request to the web shop.**



Your first local rule — the six-step method used in LAB_04

- 1 **Name the event** in one plain sentence: "a machine other than the administrator's workstation opens SSH to the server zone".
- 2 **Write the narrowest header that still catches it:** `tcp !20.20.10.10 any -> 20.20.20.0/24 22`
- 3 **Add one or two detection options** that remove noise: `flags:S;` — one connection attempt, one alert.
- 4 **Add the bookkeeping:** a clear `msg`, a `classtype`, `sid ≥ 1,000,000`, `rev:1`.
- 5 **Save in `local.rules`**, test the syntax with `-T`, restart the engine, read the "rules loaded" line.
- 6 **Validate with safe traffic** (Lesson 3) and keep the alert screenshot for Chapter 4 of the manual.



Real-life example 08-1 (1/2) — the technician checks four rules before loading them

Task

Parse the **four local rules**, check them (msg, sid ≥ 1,000,000, rev, action), then pass **six lab packets** through them in the engine's order: header first, options second.

Relation to the lesson

§ 1.2 to § 1.5 turned into a program: the same seven fields, the same option keywords, the same alert line format, the same "all or nothing".

Deliverable → used again in

[RLE08-1_rule-anatomy-table.csv](#) → **LAB_04 part 2** (the rule set used, Chapter 4) · rule 1000002 → **LAB_05** firewall rule · rules 1000003/4 → **Final Project Ch. 7** rows 3–4.

```
alert icmp any any -> $HOME_NET any (msg:"LAB rule 1 - ICMP echo request into the company network"; itype:8; classtype:not-suspicious; sid:1000001; rev:1;)

alert tcp !20.20.10.10 any -> 20.20.20.0/24 22 (msg:"LAB rule 2 - SSH connection to the server zone from a machine that is not the admin workstation"; flags:S; classtype:policy-violation; sid:1000002; rev:1;)

alert http any any -> 10.10.10.20 80 (msg:"LAB rule 3 - UNION SELECT in a request to the web shop"; flow:to_server,established; http.uri; content:"UNION SELECT"; nocase; classtype:web-application-attack; sid:1000003; rev:1;)

alert http any any -> 10.10.10.20 80 (msg:"LAB rule 4 - SQL tautology OR 1=1 in a request to the web shop"; flow:to_server,established; http.uri; content:"or 1=1"; nocase; classtype:web-application-attack; sid:1000004; rev:2;)
```



Real-life example 08-1 (2/2) — what the program printed, and the table it wrote

```
Rule sid 1000001: alert icmp any:any -> $HOME_NET:any options=5 checks=OK
Rule sid 1000002: alert tcp !20.20.10.10:any -> 20.20.20.0/24:22 options=5 checks=OK
Rule sid 1000003: alert http any:any -> 10.10.10.20:80 options=8 checks=OK
Rule sid 1000004: alert http any:any -> 10.10.10.20:80 options=8 checks=OK
Loaded 4 rules. Inspecting 6 packets...
Packet 1: lab-client pings web-01 (safe test 1) → ALERT [1:1000001:1]
Packet 2: web-01 answers the ping (echo reply) → no rule matched
Packet 3: admin workstation opens SSH to srv-files (allowed) → no rule matched
Packet 4: lab-client opens SSH to srv-files (safe test 3) → ALERT [1:1000002:1]
Packet 5: a customer searches the web shop for 'tent' → no rule matched
Packet 6: lab-client sends the tautology request (safe test 2) → ALERT [1:1000004:2]
```

3 of 6

packets produced an alert — exactly the three safe tests. The three ordinary packets stayed silent.

Deliverable: RLE08-1_rule-anatomy-table.csv

sid	rev	proto	header	detection options	classtype	used again in
1000001	1	icmp	any any -> \$HOME_NET any	itype:8	not-suspicious	LAB_04 test 1; LAB_05 tests
1000002	1	tcp	!20.20.10.10 any -> 20.20.20.0/24 22	flags:S	policy-violation	LAB_05 firewall rule; Final Project ch. 7 row 4
1000003	1	http	any any -> 10.10.10.20 80	flow; http.uri; content:UNION SELECT; nocase	web-application-attack	LAB_05 DMZ rules; ch. 7 row 3
1000004	2	http	any any -> 10.10.10.20 80	flow; http.uri; content:or 1=1; nocase	web-application-attack	LAB_04 test 2; ch. 7 row 3



Lesson 1 — videos and references

- Cisco Talos — Snort 101: "Snort 3 – Writing Rules" (official) <https://www.snort.org/resources>
- Udacity / Georgia Tech — "Snort Rules Options" <https://www.youtube.com/watch?v=v9lZthDxEUE>
- Udacity / Georgia Tech — "Snort Rule Example" <https://www.youtube.com/watch?v=SY6R-RlbKqc>
- "Writing Simple Custom Suricata Rules" https://www.youtube.com/watch?v=8h_r7brfc3E
- Suricata rules introduction <https://docs.suricata.io/en/latest/rules/intro.html>
- Suricata — adding your own rules <https://docs.suricata.io/en/latest/rule-management/adding-your-own-rules.html>
- Snort 3 rule documentation <https://docs.snort.org/rules/>

Short videos (TikTok / YouTube Shorts): search #Suricata #Snort #IDSrules and apply the four-question habit — who is the author, which version and date, does it agree with the official documentation, is the demonstration inside a laboratory. Verify every link before opening it (VirusTotal URL scanner).



Lesson 1 — five things to remember

What are the two halves of every rule?

The **header** chooses the packets; the **options**, in round brackets, say what to look for and how the alert is named.

What does [1:1000004:2] mean?

gid : sid : rev — generator 1 (the rule engine), rule 1000004, revision 2.

Why $\text{sid} \geq 1,000,000$ for your own rules?

Lower identifiers belong to distributed rule sets; a collision makes the engine refuse to start or keep only one rule.

What does `flow:to_server,established` require?

Client-to-server direction inside a connection whose TCP handshake is complete.

Header true, options false — what happens?

Nothing is logged. An alert requires every header field **and** every option to be true.



Quick check — Lesson 1

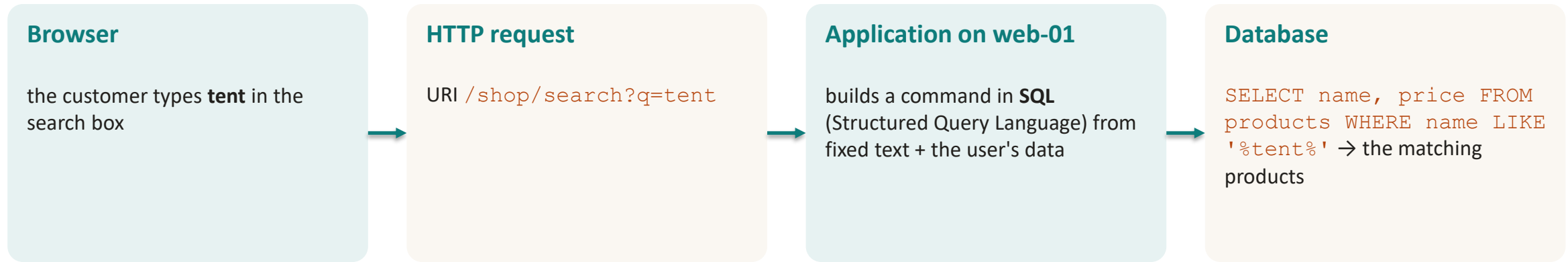
In `alert tcp !20.20.10.10 any -> 20.20.20.0/24 22 (...)` which packet can match the header?

- A** An SSH connection from 20.20.10.10 to 20.20.20.10, port 22
- B** An SSH connection from 20.20.10.77 to 20.20.20.10, port 22
- C** An SSH connection from 20.20.20.10 to 20.20.10.77, port 22
- D** A ping from 20.20.10.77 to 20.20.20.10

Check in the engine's order: protocol → the negated source → destination network and port.



How a customer's request becomes a database command



In every case the application **combines fixed command text written by the developer with data supplied by the user.**

**Web application security begins with one question:
what stops the user's data from being read as part of the command?**



Injection, defined — data and code mixed in one string

An **injection attack** occurs when data supplied by a user is placed into a command in such a way that the interpreter — the database engine, the shell, the directory server — **reads part of the data as instructions**. Root cause: **data and code are mixed in one text string** before the interpreter sees it.

SQL injection

The input becomes part of a **database query**:
bypass a login, read or change every table.
Catalogued as **CWE-89**.

OS command injection

The input becomes part of a **shell command** — a
"ping this host" page that runs **ping** with
whatever the user typed after a **;**.

XSS — Cross-Site Scripting

The input becomes part of the **HTML page** shown
to other users; their browsers run the attacker's
script.
Same root cause, different interpreter.

- **OWASP Top Ten** — injection is category **A03:2021**, one of the most serious web application risks.
- **MITRE ATT&CK T1190** — Exploit Public-Facing Application: the attacker's technique.



Worked example (1/2) — the tautology on the login form

THE VULNERABLE CODE — concatenation

```
query = "SELECT id, username, email FROM accounts WHERE username = '" + username + "'"
```

THE INPUT

```
' OR 1=1 --
```

THE COMMAND THAT REACHES THE DATABASE

```
SELECT id, username, email FROM accounts WHERE username = '' OR 1=1 --'
```

- 1 The attacker's first `'` closes the string early — the user name is now empty.
- 2 `OR 1=1` adds a condition that is **always true** (a **tautology**): the `WHERE` filter accepts every row.
- 3 `--` starts an SQL comment: the developer's final `'` is ignored instead of causing a syntax error.
- 4 Result: **every account is returned** — on a login page, the attacker is logged in as the first row, often the administrator.



Worked example (2/2) — UNION SELECT reads other tables

THE INPUT

```
' UNION SELECT id, username, email FROM accounts --
```

THE COMMAND THAT REACHES THE DATABASE

```
SELECT id, username, email FROM accounts WHERE username = '' UNION SELECT id, username, email FROM accounts --'
```

- **UNION** appends the result of a **second query** to the result of the first.
- The attacker chooses that second query freely — this is how other tables (card records, password hashes) are read through a search box meant to find tents.
- Both attack strings contain a **recognizable fragment**: `or 1=1` and `UNION SELECT`.
- That fragment is exactly what tonight's rules **1000003** and **1000004** look for in the request URI.

What the sensor raises

```
[1:1000003:1] LAB rule 3 - UNION  
SELECT in a request to the web shop  
[1:1000004:2] LAB rule 4 - SQL  
tautology OR 1=1 in a request to the  
web shop
```



Countermeasures (1/2) — the parameterized query and allow-list validation

UNSAFE — input pasted into the query text

```
"... WHERE username = '" + username + "'"
```

the typed text becomes part of the command

rows returned for ' OR 1=1 -- : 3 of 3

SAFE — parameterized query (prepared statement)

```
"... WHERE username = ?" + value bound separately
```

the command is compiled first; the input is a **value**, never code

rows returned for ' OR 1=1 -- : 0

- **Parameterized queries** — the primary countermeasure, available in every language: `?` in Python `sqlite3`, `%s` in `psycopg`, `PreparedStatement` in Java, ORM libraries.
- **Allow-list input validation** — accept only the shape the field should have (a user name of letters and digits, at most 32 characters; a quantity that is an integer 1–99). Reject the rest before it reaches any command. A complement, never a replacement.



Countermeasures (2/2) — least privilege, error handling, extra layers

Least privilege for the database account

The web shop's account can read **products** and read/write **orders** — **nothing else**.

If an injection succeeds anyway, it cannot reach the payroll table.

The **LAB_02** principle applied to a service account.

Error handling that does not leak

A database error is **never** shown to the customer — attackers read "near --: syntax error" to learn the query structure.

Log the error; show a generic message.

Extra layers where the code cannot change

A **WAF** (Web Application Firewall) in front of the application.

Framework and database kept **patched**.

They reduce exposure; they do **not** repair the vulnerability.

Reference text with code examples per language — OWASP SQL Injection Prevention Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Prevention lives inside the application. What the network sensor adds is the next slide.



Why a network sensor helps — four things the code cannot do

1

It reports the attempt

timestamp, source address, exact request — a fixed application fails silently; the sensor turns the failure into an **event** in the log.

2

It covers what is not fixed yet

a third-party plug-in, a legacy page, a supplier's appliance whose code you do not control.

3

It shows a campaign early

twenty injection alerts from one address in one minute is a scan in progress — the SOC acts before the scanner finds the one page still vulnerable.

4

It proves that a fix works

after the patch, attempts are still visible in the sensor log and the application log shows no damage.

The sensor sees the request **in transit**, before or independently of the application — on a link where HTTP is in clear text.



The limits of detection — and why it is still one of two layers

Encryption

On **HTTPS** the sensor sees only encrypted bytes. Content rules work where the sensor is placed **after TLS termination** (reverse proxy, load balancer) or on internal links that carry plain HTTP — as in the LAB_04 topology.

Evasion

Letter case (answered by **nocase**), **2=2** instead of **1=1**, comments inside keywords, encoded characters.
One **content** rule catches the textbook string and misses the variants — distributed rule sets contain **many rules per attack family**.

False positives

A forum post that legitimately contains "union select" also matches.
A detection rule is a **statement of what you want to be told about** — every alert still needs a human reading it.

Prevention in the code and detection on the wire are **two layers of the same defense-in-depth**, not alternatives — the point to make for Final Project Chapter 7, row 3: "the public web server in the DMZ is compromised".



Schematic 08-2 — why injection works, and the two layers that stop it

Why injection works, and the two layers that stop it

Customer types in the web-shop search box:

' OR 1=1 --

(harmless text — until the application decides what to do with it)

UNSAFE: input pasted into the query

```
SELECT * FROM accounts
WHERE user = ' OR 1=1 --'
```

the typed text became part of the command

Database returns EVERY row → account takeover

1 = 1 is always true, so the WHERE filter is cancelled and the whole table is dumped.

Rows returned: 3 of 3

SAFE: parameterized query

```
SELECT * FROM accounts
WHERE user = ? [value: ' OR 1=1 --']
```

the value is kept apart from the command

Database looks for a user literally named that

The input can never become SQL, so the tautology is just an unusual username.

Rows returned: 0

The IDS sees the request either way — the detection layer

Suricata reads the URL, matches content:"or 1=1", and raises:

[1:1000004:2] Web Application Attack — SQL tautology OR 1=1 in a request to the web shop

Secure coding (left) removes the vulnerability; the IDS (bottom) reports the attempt. Two layers of the same defense, not alternatives

420-D03-AB Cyber Defense · Class 8 · Lesson 2 · Cyber.SoHo Educational Hub

- **Left — unsafe:** the typed text became part of the command; **1=1** cancels the filter; **3 of 3** rows.
- **Right — safe:** the value is kept apart from the command; the tautology is just an unusual user name; **0** rows.
- **Bottom — the sensor:** reads the URL, matches **content:"or 1=1"**, raises rule 1000004 **either way**.



Real-life example 08-2 — the same three inputs through the unsafe query, the safe query and the sensor

Task

Reproduce the login form **entirely in memory**; send three inputs; show the **unsafe** query, the **safe** query, and what the **sensor** would raise.

Relation to the lesson

§ 2.3, § 2.4 and § 2.5 side by side in one run: the prevention layer and the detection layer react to the same inputs.

Deliverable → used again in

RLE08-2_injection-results.csv → Chapter 4 evidence · LAB_05 DMZ rules · Final Project Ch. 7 row 3 justification.

Input description	Value typed in login box	Rows (UNSAFE)	Rows (SAFE)	IDS alert
ordinary customer login	marie	1	1	none
injection 1 - tautology	' OR 1=1 --	3	0	sid 1000004
injection 2 - union	' UNION SELECT id, username, email FROM accounts -	3	0	sid 1000003

Conclusion: the parameterized query stops the injection at the application; the IDS reports the attempt either way — detection and secure coding are two layers of the same defense, not alternatives.



Lesson 2 — videos and references

- **Professor Messer — "SQL Injection – SY0-701 – 2.3"** (~4 min) <https://www.professormesser.com/security-plus/sy0-701/sy0-701-video/sql-injection-sy0-701/>
- **Professor Messer — "Injection Attacks – SY0-601 – 1.3"** (4 min 25 s) <https://www.youtube.com/watch?v=e4KUdwh8rGo>
- **OWASP Top Ten** <https://owasp.org/www-project-top-ten/>
- **OWASP SQL Injection Prevention Cheat Sheet**
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- **OWASP WebGoat** — deliberately vulnerable application, for your own lab only <https://owasp.org/www-project-webgoat/>
- **CWE-89** <https://cwe.mitre.org/data/definitions/89.html> · **MITRE ATT&CK T1190**
<https://attack.mitre.org/techniques/T1190/>

Short videos: search #SQLInjection #OWASP #webappsecurity and apply the four-question habit. A clip that "tests" a live public site is a counter-example, not a model. Verify every link before opening it (VirusTotal URL scanner).



Lesson 2 — five things to remember

Define an injection attack in one sentence.

User-supplied data is placed into a command so that the interpreter reads part of the data as **instructions** — data and code were mixed.

Why does ' OR 1=1 -- return every row?

The quote closes the string early; **OR 1=1** is always true, so the filter accepts every row; -- comments out the rest.

What does a parameterized query change?

The command is compiled with a placeholder first; the input is bound afterwards **as a value**, so it can never be parsed as SQL.

What does UNION SELECT give an attacker?

A **second query of their choice**, appended to the legitimate one — other tables read through a harmless-looking form.

The application is fixed. Why keep the IDS rule?

It still **reports attempts**: evidence, early warning, coverage for what is not fixed yet. Two layers, not alternatives.



Quick check — Lesson 2

Which layer prevents the injection, and which layer detects the attempt?

- A** The IDS prevents; the parameterized query detects
- B** The parameterized query prevents; the IDS detects
- C** Both prevent
- D** Neither — only a firewall on port 80 helps

Recall the conclusion printed at the end of real-life example 08-2.



Running is not detecting

Seven things that can be wrong — with no error message

1. the value of `HOME_NET`
2. the interface the engine listens on
3. whether `local.rules` is actually included
4. the syntax of one rule
5. the direction operator
6. encryption on the link
7. whether the engine was restarted after the last edit

Validation, defined

A **known, harmless event** is generated on purpose, and the **expected alert** is looked for in the log.

Only a matching alert proves that the sensor **detects** — a running process proves nothing.

What the grading checklist asks for (Final Project, Chapter 4)

"At least one captured alert corresponding to your own **safe, contained test traffic**, demonstrating that the detection system actually detects."

Tonight's lab produces it.



Safe and contained — and why a test is not an attack

Safe

Harmless to the target and to everyone else: a `ping`, one connection attempt, one HTTP request containing a recognizable text fragment. Changes no data, exploits nothing, installs nothing, leaves nothing behind.

Contained

Generated inside your isolated laboratory: from one of your virtual machines to another, on the isolated virtual network of LAB_01. Never crosses into the real network of your home, your employer or the college.

The two words matter separately: a ping is safe but not contained if it goes to a public address; a request that stays in the lab is contained but not safe if it deletes rows.

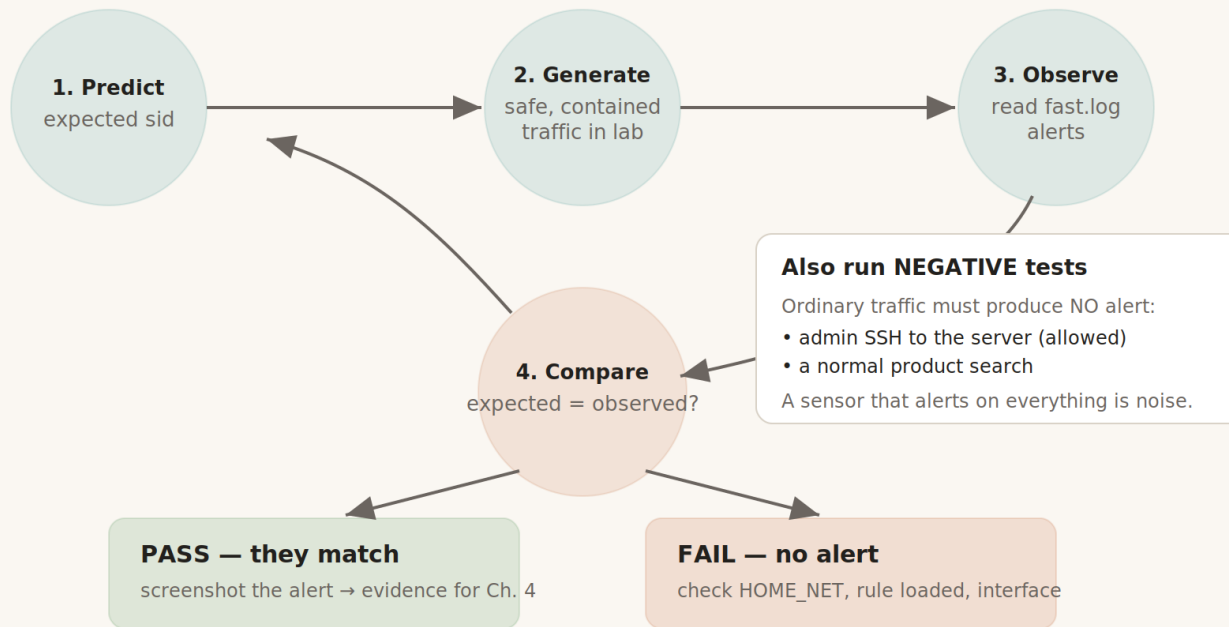
' `OR 1=1 --` sent to **your own** web-01 is a **test**. The same string sent to any website you do not own is an attempt at unauthorized access — Criminal Code s. 342.1 — whatever the intention. If an activity seems to touch a system you do not own, **stop and ask the instructor before doing anything**.



The validation loop — predict, generate, observe, compare

Proving the detector detects

Every test names its expected alert BEFORE you run it, then you compare with what the sensor logged.



420-D03-AB Cyber Defense · Class 8 · Lesson 3 · Cyber.SoHo Educational Hub

- 1 Predict** — write the expected `sid` (or "no alert") before the test.
- 2 Generate** — one action, from a named machine.
- 3 Observe** — `sudo tail -f /var/log/suricata/fast.log` (or `eve.json` with `jq`).
- 4 Compare** — `expected = observed` → **PASS**, screenshot it; else **FAIL** → § 3.6, fix, rerun.



The LAB_04 test kit — four positive tests, two negative tests

Test	Action (one command or one request, inside the lab)	From → to	Expected
T1	<code>ping -c 4 10.10.10.20</code>	lab-client → web-01	sid 1000001 (ICMP echo request)
T2	<code>ssh 20.20.20.10</code> (the attempt is enough; it may be refused)	lab-client → srv-files	sid 1000002 (SSH from a non-admin)
T3	<code>http://10.10.10.20/shop/search?q=' OR 1=1 --</code>	lab-client → web-01	sid 1000004 (tautology)
T4	<code>http://10.10.10.20/shop/search?q=' UNION SELECT id FROM accounts --</code>	lab-client → web-01	sid 1000003 (UNION SELECT)
N1	<code>ssh 20.20.20.10</code>	ws-admin → srv-files	no alert — the admin is excluded by rule 1000002
N2	<code>http://10.10.10.20/shop/search?q=tent</code>	lab-client → web-01	no alert — an ordinary search

Negative tests are not optional: a sensor that alerts on everything is noise, and the real event is buried in it. N1 and N2 prove that the rules are **specific**.



The community smoke test — and its contained version

The quick-start test (Suricata documentation)

```
curl http://testmynids.org/uid/index.html
```

The page's only content: `uid=0 (root) gid=0 (root) groups=0 (root)`

Distributed rule **2100498** "GPL ATTACK_RESPONSE id check returned root" fires on that text in the **server's reply**.

Useful on a sensor with Internet access — but **not contained**: the request leaves your lab.

The contained version (tonight's way)

Place the same **one-line text file** on your own `web-01`.

Fetch it from `lab-client`: `curl http://10.10.10.20/uid/index.html`

The rule looks at the **response body**, not at the address — it fires identically.

Everything stays between your two virtual machines.

<https://docs.suricata.io/en/latest/quickstart.html> · <http://testmynids.org/uid/index.html>



Reload the rules and read the start-up line

Suricata (installed as a service)

```
sudo suricata -T -c /etc/suricata/suricata.yaml
sudo systemctl restart suricata
live reload instead: sudo suricatasc -c reload-rules
read: 4 rules successfully loaded, 0 rules failed
```

Snort 3

```
snort -T -c /usr/local/etc/snort/snort.lua -R
local.rules
then start the engine again on the interface with -i
-A alert_fast prints alerts to the console
```

- A rule that **fails to load** is printed with its line number and the reason.
- The three usual reasons: a missing **semicolon**, an unbalanced **quotation mark**, a **duplicated identifier**.
- Read the two numbers of the start-up line **every time** — they are also evidence for Chapter 4 (once per chapter).



When the expected alert does not appear — the troubleshooting order

- 1 **Is the rule loaded?** Start-up count, or `-T`. If it failed, the reason is printed.
- 2 **Does `HOME_NET` contain the addresses of the test?** Compare with the LAB_01 zoning table.
- 3 **Is the sensor listening on the interface that carries the test?** `-i` or the `af-packet` section; `sudo tcpdump -ni eth1 host 10.10.10.20` shows whether the packets are visible there.
- 4 **Direction and port.** Re-read the header against the test: who started the connection, which port.
- 5 **Encryption.** A content rule cannot match inside HTTPS — test on the plain HTTP port.
- 6 **Was the engine restarted after the edit?** Compare the start-up timestamp with the time of the edit.
- 7 **Is the test really generating the traffic?** Browser cache, or a LAB_03 host firewall blocking the ping — confirm with `tcpdump` on the client.



What counts as evidence in Chapter 4

1 · The action

The command or the request **as typed**, with the **clock of the client** visible.

2 · The alert

The `fast.log` line whose **timestamp, sid and addresses match** the test.

3 · The table row

Test · expected · observed · **verdict** — the evidence table of real-life example 08-3.

- One cropped screenshot per test, with a one-line caption; the "**rules loaded**" line once per chapter.
- Laboratory addresses and laboratory accounts only — **no real data, no personal data** (documentation standard § 2.7).
- A screenshot of the sensor process in `ps` proves that it **runs**; it does not prove that it **detects**.



Real-life example 08-3 — the validation plan, and the evidence table it writes

Task

Six tests; the **expected** result written before the run; simulate, observe, **compare**; PASS or FAIL.

Relation to the lesson

The validation loop of § 3.3 executed six times, negative tests included, evidence standard of § 3.7 as output.

Deliverable → used again in

RLE08-3_validation-evidence.csv →
LAB_04 part 2 validation · **LAB_05**
allowed/blocked tests · **Final Project Ch. 4.**

Test	What is tested	How to generate it (inside the lab)	Expected	Observed	Verdict
T1	ping the web server from the lab client	on lab-client: ping -c 4 10.10.10.20	1000001	1000001	PASS
T2	SSH to the file server from a non-admin workstation	on lab-client: ssh 20.20.20.10	1000002	1000002	PASS
T3	tautology string to the web shop search box	browser: /shop/search?q=' OR 1=1 --	1000004	1000004	PASS
T4	UNION SELECT string to the web shop search box	browser: /shop/search?q=' UNION SELECT ...	1000003	1000003	PASS
N1	SSH to the file server FROM the admin workstation	on ws-admin: ssh 20.20.20.10	none	none	PASS
N2	ordinary product search on the web shop	browser: /shop/search?q=tent	none	none	PASS

6 of 6 tests passed — sensor validated: detection proven for every rule and quiet on normal traffic. Tonight, replace the "Observed" column with what your own fast.log shows.



Lesson 3 — videos and references

- **Cisco Talos — Snort 101: "Snort 3 – Logging (with labs)"** <https://www.snort.org/resources>
- **Suricata quick start** — the "Alerting" paragraph is the shortest authoritative description of the validation loop
<https://docs.suricata.io/en/latest/quickstart.html>
- **testmynids.org** — the one-line public test page (host a copy on your own web server for a contained test)
<http://testmynids.org/uid/index.html>
- **Professor Messer — "Intrusion Prevention – SY0-701 – 4.5"** <https://www.professormesser.com/security-plus/sy0-701/sy0-701-video/>
- **NIST SP 800-94** — guide to intrusion detection and prevention systems, including validation of the deployment
<https://csrc.nist.gov/pubs/sp/800/94/final>

Short videos: search #testmynids #Suricata #fastlog #SOC and apply the four-question habit. Any clip that "tests" a sensor by attacking a machine the author does not own is a counter-example. Verify every link before opening it (VirusTotal URL scanner).



Lesson 3 — five things to remember

Running versus detecting?

Running: the process is up and sees packets.

Detecting: a known, safe event produced the expected alert in the log. Only the second is evidence.

What makes a test safe and contained?

Safe: harmless to the target. **Contained:** inside your isolated lab, your machine to your machine.

The four steps of the validation loop?

Predict the sid before the test · **Generate** one action · **Observe** fast.log · **Compare** → PASS or FAIL.

Why run negative tests?

Ordinary traffic must produce **no alert**. A sensor that alerts on everything is noise; the real event is buried in it.

Expected alert missing — first three checks?

(1) rule **loaded** · (2) **HOME_NET** contains the test addresses · (3) sensor listening on the **interface** that carries the test.



Quick check — Lesson 3

T2 expects sid 1000002 but fast.log shows nothing, while N1 (the administrator's SSH) correctly produces no alert. Most likely cause?

- A The sensor is not running at all
- B HOME_NET is wrong
- C T2 was run from ws-admin (20.20.10.10), the one address the rule excludes
- D The rule needs nocase

Read the source address of rule 1000002 and the machine used for the test.



LAB_04 part 2 — tonight's work (1/2)

Objective: turn the sensor installed in part 1 into a **validated** detection system — "enable a small provided rule set, generate harmless test traffic inside your own lab, and capture the resulting alerts."

- 1 **Enable the provided rule set** and add the four local rules of Lesson 1 (or your own, by the six-step method). Test with `-T`, restart or reload, read the "rules loaded" line.
- 2 **Predict** — fill the *expected* column of your evidence table before any test (Lesson 3 § 3.3).
- 3 **Generate the safe, contained tests** — T1 to T4 and the two negative tests — one action at a time, from the named machine.
- 4 **Capture the alerts** — `fast.log` lines (and `eve.json` events if you use them), the screenshots of § 3.7 — and write **Chapter 4** of the manual tonight, in the supervised work time, following the chapter template.



LAB_04 part 2 — the deliverable, and the rules that never change (2/2)

Deliverable — Chapter 4 "Active defense" (folder CH4_detection/)

Installation notes (from part 1, updated with the rule-file and HOME_NET settings).

The rule set used — provided rules + your local rules, with the rule-anatomy table (RLE08-1_rule-anatomy-table.csv).

Alert screenshots from your own safe test traffic, with the evidence table (RLE08-3_validation-evidence.csv) filled with **your** observed values.

Grading: Final Project Ch. 4 = **2.0 points** (exists and works 1.0 · validated 0.5 · justified 0.5); manual Ch. 4 = **1.0 point**.

Rules that never change

Isolated laboratory only; your virtual machines only; snapshots before you edit the sensor.

The injection strings go to **your own** web-01 and nowhere else.

AI tools used? The **LLM and the exact prompts** go in section 4.6 of the chapter — otherwise the sentence `"No AI tools were used in this chapter."`

"The model drafts. The analyst decides. The analyst's name goes on the record."

Due date and submission channel: **as stated in the LAB_04 guide** distributed at Class 7.



Wrap-up — the three sentences to remember

- 1 A rule is a **header** that chooses the packets and a list of **options** that says what to look for; the engine alerts **only if every condition is true**, and the alert line names the rule by `[gid:sid:rev]`.
- 2 Injection works because **data and code are mixed in one string**; the **parameterized query** keeps them apart, and the sensor **reports the attempt either way** — prevention and detection are two layers of one defense.
- 3 A sensor that runs is not a sensor that detects: you **predict, generate safe contained traffic, observe, compare**, keep the screenshots, and prove with **negative tests** that the rules are specific.

Next: LAB_04 part 2 briefing (7:00–7:30) · break (7:30–7:45) · supervised lab work (7:45–9:00)



Hyperlinks used in this lecture (1/2)

ENGINES AND DOCUMENTATION

- **Suricata** — Open-source IDS/IPS engine used by most of the class <https://suricata.io/>
- **Suricata rules introduction** — Rule format: header fields and option keywords (Lesson 1 reference) <https://docs.suricata.io/en/latest/rules/intro.html>
- **Suricata - adding your own rules** — Where local.rules goes and how to see that it loaded <https://docs.suricata.io/en/latest/rule-management/adding-your-own-rules.html>
- **Suricata quick start** — First validation test, Alerting paragraph (Lesson 3) <https://docs.suricata.io/en/latest/quickstart.html>
- **OISF** — Foundation that maintains Suricata <https://oisf.net/>
- **Snort** — Snort 3 (Cisco Talos) <https://www.snort.org/>
- **Snort 3 rule documentation** — Snort rule syntax (differences noted in Lesson 1) <https://docs.snort.org/rules/>
- **Snort resources (Snort 101 videos)** — Official video series: installation, writing rules, logging <https://www.snort.org/resources>
- **Emerging Threats Open rules** — Distributed rule set that owns identifiers below 1,000,000 <https://rules.emergingthreats.net/>
- **testmynids.org test page** — One-line public test page used by the Suricata quick start <http://testmynids.org/uid/index.html>
- **Security Onion** — Free platform packaging Suricata with log tools (optional) <https://securityonion.net/>

WEB APPLICATION SECURITY

- **OWASP** — Open Worldwide Application Security Project <https://owasp.org/>
- **OWASP Top Ten** — Injection is category A03:2021 <https://owasp.org/www-project-top-ten/>
- **OWASP SQL Injection Prevention Cheat Sheet** — Countermeasures of Lesson 2 with code examples https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- **OWASP WebGoat** — Deliberately vulnerable application for practice inside your own lab <https://owasp.org/www-project-webgoat/>
- **CWE-89** — Catalogue entry for SQL injection <https://cwe.mitre.org/data/definitions/89.html>
- **MITRE ATT&CK T1190** — Exploit Public-Facing Application <https://attack.mitre.org/techniques/T1190/>

FRAMEWORKS AND TOOLS

- **CIS** — Center for Internet Security, Critical Security Controls <https://www.cisecurity.org/>
- **NIST SP 800-94** — Guide to intrusion detection and prevention systems <https://csrc.nist.gov/pubs/sp/800/94/final>
- **Wireshark** — Packet analysis when a test fails <https://www.wireshark.org/>
- **curl** — Send one exact HTTP request from the command line <https://curl.se/>
- **Python** — Language of the three examples <https://www.python.org/>
- **SQLite** — In-memory database engine used by RLE 08-2 <https://www.sqlite.org/>



Hyperlinks used in this lecture (2/2)

VIDEOS

- **Udacity / Georgia Tech - Snort Rules Options** — Short clip on rule options (verify playback) <https://www.youtube.com/watch?v=v91ZthDxEUE>
- **Udacity / Georgia Tech - Snort Rule Example** — Short clip walking through one rule (verify playback) <https://www.youtube.com/watch?v=SY6R-RlbKqc>
- **Writing Simple Custom Suricata Rules** — One custom rule for an HTTP attack in a capture file (verify length) https://www.youtube.com/watch?v=8h_r7brfc3E
- **Professor Messer - SQL Injection SY0-701 2.3 (page)** — Course page with the embedded video <https://www.professormesser.com/security-plus/sy0-701/sy0-701-video/sql-injection-sy0-701/>
- **Professor Messer - SQL Injection SY0-701 2.3 (video)** — About 4 minutes (verify playback) <https://www.youtube.com/watch?v=qFUOLkEk8AQ>
- **Professor Messer - Injection Attacks SY0-601 1.3 (page)** — Course page with the embedded video <https://www.professormesser.com/security-plus/sy0-601/sy0-601-video/injection-attacks/>
- **Professor Messer - Injection Attacks SY0-601 1.3 (video)** — 4 min 25 s, whole injection family <https://www.youtube.com/watch?v=e4KUdwh8rGo>
- **Professor Messer - SY0-701 course index** — Intrusion prevention video for the Lesson 3 recap <https://www.professormesser.com/security-plus/sy0-701/sy0-701-video/>

LAW AND REGULATION

- **Criminal Code of Canada** — s. 342.1 and s. 430(1.1) <https://laws-lois.justice.gc.ca/eng/acts/C-46/>
- **Commission d'accès à l'information du Québec** — Québec Law 25 <https://www.cai.gouv.qc.ca/>
- **Office of the Privacy Commissioner of Canada** — PIPEDA <https://www.priv.gc.ca/>
- **GDPR (EUR-Lex)** — Regulation (EU) 2016/679 <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- **US CFAA 18 U.S.C. § 1030** — Computer Fraud and Abuse Act <https://www.law.cornell.edu/uscode/text/18/1030>
- **EU Directive 2013/40/EU** — Attacks against information systems <https://eur-lex.europa.eu/eli/dir/2013/40/oj>
- **Spain Ley Orgánica 10/1995** — Código Penal <https://www.boe.es/buscar/act.php?id=BOE-A-1995-25444>

VERIFICATION

- **VirusTotal URL scanner** — Check any link before opening it <https://www.virustotal.com/gui/home/url>



Glossary of abbreviations (1/2)

Abbreviation	Expansion
AEC	Attestation d'études collégiales
AI	Artificial Intelligence
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge (MITRE)
AWS	Amazon Web Services
CFAA	Computer Fraud and Abuse Act (United States, 18 U.S.C. § 1030)
CIDR	Classless Inter-Domain Routing
CIS	Center for Internet Security
CSV	Comma-Separated Values
CWE	Common Weakness Enumeration
DMZ	Demilitarized Zone
EDT	Eastern Daylight Time
ET	Emerging Threats (open rule set)
EU	European Union

Abbreviation	Expansion
EVE	Extensible Event Format (Suricata eve.json)
GDPR	General Data Protection Regulation
GID	Generator Identifier
HIDS	Host-based Intrusion Detection System
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ICMP	Internet Control Message Protocol
IDS	Intrusion Detection System
IP	Internet Protocol
IPS	Intrusion Prevention System
JSON	JavaScript Object Notation
LDAP	Lightweight Directory Access Protocol



Glossary of abbreviations (2/2)

Abbreviation	Expansion
LLM	Large Language Model
MCQ	Multiple-Choice Question
MSQ	Multiple-Select Question
NIDS	Network-based Intrusion Detection System
NIST	National Institute of Standards and Technology
OISF	Open Information Security Foundation
ORM	Object-Relational Mapping
OWASP	Open Worldwide Application Security Project
PIPEDA	Personal Information Protection and Electronic Documents Act
PNG	Portable Network Graphics
RLE	Real-Life Example
SID	Signature Identifier
SOC	Security Operations Center

Abbreviation	Expansion
SQL	Structured Query Language
SSH	Secure Shell
SVG	Scalable Vector Graphics
SYN	Synchronize (TCP flag of the first packet of a connection)
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
WAF	Web Application Firewall
XML	Extensible Markup Language
XSS	Cross-Site Scripting



Questions — then LAB_04 part 2

7:00–7:30 LAB_04 part 2 briefing · 7:30–7:45 break · 7:45–9:00 supervised lab work

Isolated laboratory only · your own virtual machines only · snapshots first · injection strings to your own web-01 only

AI tools used? LLM and exact prompts in the chapter — otherwise "No AI tools were used in this chapter."

Legal Notice, Disclaimer, and Acceptable Use Policy — Cyber Defense (420-D03-AB) — Version 1.0, effective August 29, 2026 — accepted by use and by the session e-signature.

Criminal Code of Canada s. 342.1, s. 342.2, s. 430(1.1) · Québec Law 25 · PIPEDA · GDPR · US CFAA 18 U.S.C. § 1030 · EU Directive 2013/40/EU · Spain Ley Orgánica 10/1995.

Educational purpose only · no warranty · not professional advice · © 2026 Cyber.SoHo — All rights reserved · no affiliation with any product or platform named.



Educational Hub © 2026 | From the Industry to the Classroom — Building the IT's Future, Today and Together !

